



ECOPLATE PROJECT DOCUMENTATION

Full Project Explanation Report

A complete explanatory PDF for the EcoPlate food donation platform, covering purpose, architecture, workflows, data model, security, setup, and important code files.

Project

EcoPlate food donation and charity pickup platform

Generated

2026-06-05

Stack

Next.js, React, TypeScript, Firebase, Tailwind CSS,
Gemini, Brevo

Source

Final Project repository files

Table of Contents

1. Executive Summary	8. Security and Access Control
2. Problem and Project Aim	9. Codebase Structure
3. Users and Role Permissions	10. Important File Responsibilities
4. System Architecture	11. Setup and Configuration
5. Main User Workflows	12. Testing and Build Checks
6. Feature Explanation	13. Limitations and Future Improvements
7. Data Model	14. Conclusion

Executive Summary

EcoPlate is a production-style food donation web application that connects restaurants with charities. Restaurants can post surplus food, charities can request pickup quantities, and admins moderate accounts and advertisements. The system is built with Next.js App Router, React, TypeScript, Tailwind CSS, Firebase Authentication, Cloud Firestore, Firebase Storage, Gemini, and Brevo email.

3

core user roles

4

main Firestore collections

18

route/API surfaces

Next.js 16

front-end framework

This document explains the project as a final-project style report. It covers the purpose, architecture, functional modules, role workflows, data model, security rules, important files, setup steps, and recommended improvements.

Problem and Project Aim

Restaurants often have usable surplus food, while charities need dependable ways to find and collect donations. Without a structured platform, pickup coordination can become slow, manual, and difficult to audit. EcoPlate addresses this by providing a role-based digital workflow for posting food, requesting quantities, managing pickup decisions, and notifying approved charities.

Project Objectives

- Reduce food waste by making available restaurant donations visible to approved charities.
- Protect operational access through Firebase Authentication and admin approval gates.
- Track donation quantities so charities cannot request more than the remaining amount.
- Support pickup coordination with location, expiry, and request-status information.
- Allow restaurants to publish paid advertisements after admin review.
- Provide AI guidance through an EcoPlate chatbot that can answer both general and database-aware questions.

Scope

The project focuses on food donation coordination, account approval, advertisement moderation, and basic AI assistance. It does not implement full payment processing, logistics dispatch, advanced inventory forecasting, or native mobile apps; those are suitable future enhancements.

Users and Role Permissions

EcoPlate has three roles: restaurant, charity, and admin. Restaurant and charity accounts are public registration roles, but they start as pending accounts. Admin accounts are deliberately not self-service; a user document must be promoted to admin inside Firestore.

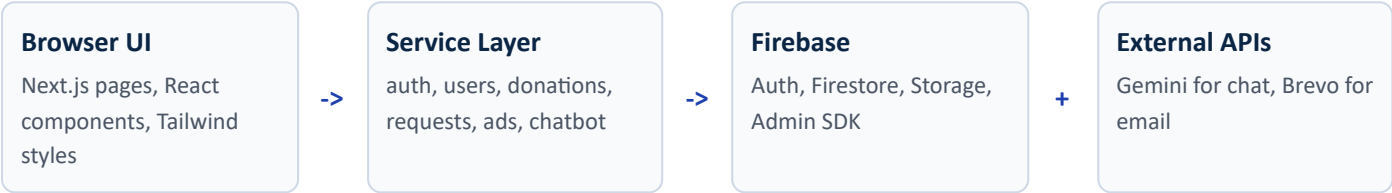
Role	Main Permissions	Restricted From
Restaurant	Create donations, edit own donations, review requests for own donations, submit and track ads.	Admin dashboard, approval queues, other restaurants' private records.
Charity	Browse available donations, request pickup quantities, track own requests, use the chatbot.	Creating donations, approving requests, managing advertisements.

Role	Main Permissions	Restricted From
Admin	Approve/reject users, view platform stats, moderate ads, manage users and records.	Public self-registration; admin role is assigned manually for security.

Approval gate: login can succeed at Firebase level while EcoPlate still blocks the user if their Firestore profile is pending or rejected.

System Architecture

EcoPlate uses a client-heavy Next.js App Router structure with Firebase services as the main backend. Most CRUD operations are performed from client components through Firestore service modules. Server-side API routes are used for chatbot requests and donation notification emails.



Technology Stack

Layer	Technology	Purpose
Application framework	Next.js 16 App Router	Routing, pages, API routes, production build.
UI runtime	React 19 and TypeScript	Interactive client components with typed models.
Styling	Tailwind CSS	Responsive design, utility classes, reusable visual patterns.
Authentication	Firebase Authentication	Email/password and Google sign-in.
Database	Cloud Firestore	Users, donations, requests, advertisements, and realtime listeners.
Storage/uploads	Firebase Storage and base64 helpers	Ad images plus base64 image handling for avatars and donations.
AI assistant	Gemini API	General help and database-aware answers through controlled prompts.
Email	Brevo Transactional Email API	New donation notification emails to approved charities.
Maps	Leaflet and React Leaflet	Donation pickup selection and route/location display.

Main User Workflows

Restaurant Donation Flow

1. A restaurant registers and waits for admin approval.
2. After approval, the restaurant creates a donation with quantity, pickup time, expiry, image, and location.
3. Approved charities browse available donations and submit pickup requests.
4. The restaurant approves or rejects requests.
5. When a request is approved, the remaining quantity is reduced transactionally. If nothing remains, the donation becomes completed.

Charity Pickup Flow

1. A charity registers with certificate data and waits for admin approval.
2. The charity dashboard subscribes to available donations and the charity's request list.
3. The charity opens a donation detail page, checks location guidance, enters requested quantity, and sends a message.
4. The request appears as pending until the restaurant approves or rejects it.
5. Approved requests become a pickup schedule for the charity.

Admin Moderation Flow

1. An admin account is created by changing a Firestore user role to admin.
2. The admin dashboard summarizes users, donations, requests, and ads.
3. Pending restaurant and charity accounts are approved or rejected.
4. Advertisement submissions are reviewed, payment status is marked, and qualified ads are published.
5. Admin rules also permit cleanup and management actions that ordinary users cannot perform.

Feature Explanation

Authentication and Approval

Users register as either restaurant or charity. The registration page requires organization details and a license or certificate upload. New accounts are written to the users collection with approvalStatus set to pending, then the user is signed out until an admin approves the account.

Donation Management

Restaurants create donation posts with food details, total quantity, remaining quantity, pickup information, expiry information, optional image, and map coordinates. Available donations are filtered so expired posts do not remain visible to charities.

Request Management

Charities request a specific quantity from an available donation. The request service prevents duplicate active requests and uses Firestore transactions during approval so the remaining quantity and request status update together.

Advertisement Publishing

Restaurants submit ads with pending payment and pending review status. Public ad surfaces only show records where paymentStatus is paid and status is published. Admins control the review, payment, and publishing states.

AI Chatbot

The chatbot detects the user's language mode and intent. General questions go directly to Gemini. Database questions first collect role-appropriate facts from Firestore, then Gemini formats the answer. Role restrictions prevent a user from asking for data they should not see.

Email Notifications

The donation email API sends new-donation notifications to approved charities. It prefers the Firebase Admin SDK for authoritative charity lookup and can fall back to a client-provided charity list during local development.

Data Model

The core database model is intentionally simple and suitable for a student or early production project.

Collection	Purpose	Important Fields	Status Values
users	Stores Firebase Auth profile data and EcoPlate role metadata.	id, name, email, phone, address, role, avatar64, licenseFile, approvalStatus, isApproved, createdAt	role: restaurant, charity, admin; approvalStatus: pending, approved, rejected
donations	Stores food donation posts created by restaurants.	restaurantId, foodName, quantity, totalQuantity, remainingQuantity, requestedQuantity, pickupLocation, pickupTime, expiryDate, expiryTime, expiresAt, status, image64, latitude, longitude, locationText, createdAt	available, requested, completed, cancelled, expired
requests	Stores charity pickup requests for donation posts.	donationId, restaurantId, charityId, charityName, message, requestedQuantity, status, createdAt	pending, approved, rejected
ads	Stores restaurant advertisement submissions.	title, description, imageUrl, restaurantId, contactNumber, paymentStatus, status, createdAt	paymentStatus: pending, paid; status: pending, approved, rejected, published

Security and Access Control

Security is enforced in two places: UI route guards and Firebase security rules. The UI improves user experience by redirecting users away from pages they cannot use. Firestore and Storage rules are still the authoritative protection layer.

Firestore Rules Summary

- Only signed-in users can read user profiles; users may update their own non-role, non-approval fields, while admins can manage all users.
- Donation reads require an approved account. Restaurants can create and update their own donations; admins have broader management rights.
- Request reads are limited to admins, the charity that created the request, or the restaurant that owns the related donation.

- Only charities can create requests. Restaurants can update request decisions for their own donations without changing immutable request fields.
- Ads are public only when paid and published. Restaurants can create and edit their own ads without changing protected review/payment fields. Admins moderate ads.

Storage Rules Summary

The storage rules allow public reading of restaurant ad images while writes are limited to authenticated users uploading image content under their own restaurant folder.

Security note: environment variables are documented by key name only. Secret values from .env.local should never be copied into reports, screenshots, Git commits, or public deployments.

Codebase Structure

The project follows a readable Next.js layout where routes, components, services, libraries, styles, and shared types are separated.

src/	
app/	Next.js pages and API routes
components/	Reusable UI and workflow components
lib/	Firebase, auth, image, email, Gemini, chat helpers
services/	Firestore-facing data access modules
styles/	Global Tailwind/CSS styles
templates/email/	Donation email HTML template
types/	Shared TypeScript data models
public/	Static assets
firestore.rules	Firestore role and ownership rules
storage.rules	Firebase Storage image rules

Route/API	Purpose	Explanation
/	Public landing page	Shows the EcoPlate pitch and public paid/published advertisements.
/register	Registration	Creates restaurant or charity accounts with profile, avatar, and license/certificate data.
/login	Login	Authenticates approved users through email/password or Google sign-in.
/dashboard	Role dashboard	Redirects admins to /admin and shows restaurant or charity metrics for approved users.
/donations	Donation list	Restaurants manage their posts; charities browse available donations.
/donations/new	New donation	Restaurant-only form for creating a donation with quantity, image, expiry, and map location.
/donations/[id]	Donation details	Displays one donation, supports restaurant editing/status changes and charity requests.
/requests	Request workflow	Restaurants approve/reject incoming requests; charities track their own request status.
/ads	Public advertisements	Displays ads after payment and publishing approval.

Route/API	Purpose	Explanation
/ads/new	Create ad	Restaurant-only advertisement submission flow.
/ads/my	My ads	Restaurant ad tracking page for payment, review, and publishing state.
/admin	Admin dashboard	Admin-only overview for users, donations, requests, ads, and destructive controls.
/admin/pending-approvals	Approval queue	Admin-only review of pending restaurant and charity accounts.
/admin/ads	Ad moderation	Admin-only payment, approval, rejection, and publishing controls.
/chat	AI assistant	EcoPlate chatbot page backed by a Next.js API route and Gemini.
/profile	Profile	Authenticated user profile view and edit page.
/api/chat	Chat API	Verifies Firebase ID token, gathers facts when needed, and asks Gemini for a reply.
/api/send-donation-email	Email API	Sends new-donation emails to approved charities through Brevo.

Important File Responsibilities

Service and Library Files

File	Responsibility
src/lib/auth.ts	Wraps Firebase Authentication, creates app profiles, blocks pending/rejected accounts, and redirects roles.
src/services/users.ts	Manages user profiles, role validation, admin approval/rejection, pending-user subscriptions, and user counts.
src/services/donations.ts	Creates and updates donation records, filters available donations, subscribes to real-time donation data, and marks expired posts.
src/services/requests.ts	Creates pickup requests and uses Firestore transactions to approve or reject requests without corrupting quantities.
src/services/ads.ts	Creates restaurant ads, lists restaurant/admin/public ads, and updates payment/review/publish states.
src/services/chatbot.ts	Detects chat intent, enforces role restrictions, reads Firestore facts through REST, and sends context to Gemini.
src/lib/gemini.ts	Holds Gemini prompt behavior for general and database-backed chatbot answers.
src/lib/firebase.ts	Initializes the client Firebase app, Authentication, Firestore, and Storage from environment variables.
src/lib/firebase-admin.ts	Lazily initializes Firebase Admin for privileged server-side routes such as email notifications.
src/lib/image.ts	Validates and processes uploaded images for avatars, donations, and advertisements.

Component Files

Component	Responsibility
ProtectedRoute.tsx	Client-side guard for login, allowed roles, and approval requirements.
AppShell.tsx, Navbar.tsx, Sidebar.tsx	Application frame, navigation, workspace shell, and authenticated navigation state.
WorkspaceUI.tsx	Reusable dashboard cards, page headers, loading skeletons, and state panels.
DonationForm.tsx, DonationCard.tsx	Donation creation/edit form and reusable donation display card.
DonationLocationMap.tsx, LocationPickerMap.tsx	Leaflet map components for pickup selection and route display.
RequestCard.tsx	Reusable view for request details and restaurant approval actions.
AdForm.tsx, AdCard.tsx, HomepageAds.tsx	Advertisement submission, card display, and landing-page ad carousel.
ImageUpload.tsx, AvatarUpload.tsx, FileUpload.tsx	Reusable file inputs with preview, validation, and base64 handling.
ChatBox.tsx	Chat interface that sends messages to /api/chat and displays the assistant reply.
ExpirationService.tsx	Background client helper that can clean up expired donations for allowed users.

Configuration Files

File	Responsibility
package.json	Project scripts and dependencies. Main scripts are dev, build, start, and typecheck.
next.config.ts	Next.js configuration, including remote image allowances if used by the app.
tailwind.config.ts	Tailwind theme extension and source scanning paths.
postcss.config.js	PostCSS setup for Tailwind processing.
tsconfig.json	TypeScript compiler settings and path aliases.
.gitignore	Prevents generated dependencies, build output, environment files, and local artifacts from entering Git.
firebase.rules	Database authorization rules for users, donations, requests, and ads.
storage.rules	Firebase Storage rules for advertisement image uploads.

Setup and Configuration

Local Setup

```
npm install
npm run dev
npm run typecheck
npm run build
```

After starting the development server, open `http://localhost:3000`.

Required Environment Variables

The project uses these keys. Keep actual values private and store them in `.env.local` or the deployment provider's environment settings.

NEXT_PUBLIC_FIREBASE_API_KEY	NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
NEXT_PUBLIC_FIREBASE_PROJECT_ID	NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID	NEXT_PUBLIC_FIREBASE_APP_ID
NEXT_PUBLIC_SITE_URL	GEMINI_API_KEY
FIREBASE_PROJECT_ID	FIREBASE_CLIENT_EMAIL
FIREBASE_PRIVATE_KEY	BREVO_API_KEY
BREVO_SENDER_EMAIL	BREVO_SENDER_NAME

Firebase Setup Checklist

1. Create a Firebase project.
2. Enable Email/Password sign-in and configure Google sign-in if required.
3. Create a Cloud Firestore database.
4. Create a Firebase Storage bucket.
5. Copy Firebase web app config values into the public Firebase environment keys.
6. Deploy `firestore.rules` and `storage.rules`.
7. Register one restaurant and one charity through the UI.
8. Manually promote an account to admin by changing that user's role in Firestore.
9. Configure Gemini and Brevo keys for chatbot and email features.

Testing and Build Checks

The project includes TypeScript and production build checks. These checks verify static typing and whether Next.js can build the app successfully.

Command	Purpose
<code>npm run typecheck</code>	Runs TypeScript without emitting files. Use after service, type, and component changes.
<code>npm run build</code>	Creates a production Next.js build and catches framework/runtime build issues.
<code>npm run dev</code>	Runs the local development server for manual UI verification.

Suggested Manual Test Cases

- Register a restaurant and confirm it cannot access protected pages until admin approval.
- Approve the restaurant from the admin approval queue.
- Create a donation with quantity, image, expiry, and map location.
- Register and approve a charity, then request part of the available quantity.

- Approve the request and confirm remainingQuantity decreases correctly.
- Submit a restaurant ad, then approve, mark paid, and publish it from the admin ad page.
- Ask the chatbot for general guidance and role-specific database facts.
- Run typecheck and build before final submission.

Limitations and Future Improvements

Current Limitations

- Payment handling is represented by paymentStatus fields rather than a real payment gateway.
- Email notifications depend on Brevo configuration and approved charity email data.
- Some image data is stored as base64, which is simple but less scalable than fully object-storage-based uploads.
- The chatbot reads a limited number of Firestore records for safety and simplicity.
- Restaurant request real-time support is partially simplified compared with a fully denormalized backend.

Recommended Enhancements

- Integrate a real payment provider for advertisement payments.
- Move all user-uploaded images to Firebase Storage and store only URLs in Firestore.
- Add Cloud Functions for donation expiry cleanup, notification automation, and denormalized counters.
- Add formal unit/integration tests for service functions and role-based access behavior.
- Add analytics for meals rescued, food categories, pickup completion rates, and charity impact.
- Improve logistics with route optimization, pickup reminders, and status timestamps.
- Add audit logs for admin approvals, ad publishing, and request decisions.

Conclusion

EcoPlate is a clear final-project example of a role-based, full-stack web application. It combines a modern Next.js interface with Firebase authentication, Firestore data modeling, security rules, maps, email notifications, and AI assistance. The strongest parts of the implementation are its separated service layer, typed data models, approval workflow, transaction-safe request approval, and practical admin moderation tools.

With payment integration, deeper automated tests, storage-based media handling, and server-side automation through Cloud Functions, the project can grow from a strong academic prototype into a more scalable operational platform.

EcoPlate project explanation report. Generated from repository source files; no source-code behavior was changed.